QUALITY ASSURANCE TESTING DOCUMENTATION

(QATD)

UMHackathon 2026

umhackathon@um.edu.my

© UMHackathon 2026

Email: umhackathon@um.edu.my

Table of Content

Document Control **3**

PRELIMINARY ROUND (Test Strategy & Planning) **3**

1. Scope & Requirements Traceability 3
2. Risk Assessment & Mitigation Strategy 3
3. Test Environment & Execution Strategy 5
4. CI/CD Release Thresholds & Automation Gates 5
5. Test Case Specifications (Drafts) 6
6. Test Strategy & Plan Sign-Off 7

© UMHackathon 2026

Email: umhackathon@um.edu.my

Document Control

Field	Detail
System Under Test (SUT)	*RetOps — UMHackathon 2026, Team Hartalis*
Team Repo URL	*https://github.com/SteveNoJob/Hartalis*
Project Board URL	*[Insert link to your Jira/Trello Projects board]*
Live Deployment URL (optional for preliminary round)	*[Insert link to the live hosted application]*

Objective:

The primary objective is to ensure that RetOps reliably ingests transactional and inventory data uploaded by business owners, produces accurate GLM-powered demand forecasts and reorder recommendations, handles malformed inputs and GLM failures gracefully, and meets response time thresholds under concurrent load — validated with CI/CD checkpoints throughout development.

PRELIMINARY ROUND (Test Strategy & Planning)

1. Scope & Requirements Traceability

*This section aligns the testing connected back to specific user requirements via **Requirement Traceability Matrix".** So, it ensures every test has been conducted to prevent bugs/failure in products for that specific requirements and unplanned feature creep.*

1.1 In-Scope Core Features

- **Data Upload & Parsing: CSV/Excel upload, pandas validation, data quality warning surfaced to user**
- **Demand Forecasting: 7-day per-item GLM forecast with weather + holiday enrichment and written reasoning output**
- **Reorder Recommendation: GLM shopping list with quantities, timing, and Malay/English plain-language explanation per item**
- **Inventory Optimisation: GLM interpretation of inconsistently formatted inventory files, high-velocity item identification**
- **What-If Scenario Planning: Free-text scenario input adjusts forecast and recommendation output with visible reasoning**

© UMHackathon 2026

Email: umhackathon@um.edu.my

1.2 Out-of-Scope

- Real-time POS integration
- Automated purchase order generation / supplier APIs
- Multi-user accounts and role-based access control
- Mobile application (iOS/Android) - File formats beyond CSV and Excel

2. Risk Assessment & Mitigation Strategy

*[e.g. Quality Assurance risks are anticipatory. So, identify the technical risk associated with you application requirements, architecture. Now, the risk needs to be evaluated using the 5*5 Risk assessment Matrix. So, Risk Score = Likelihood * Severity.]*

Technical Risk	Likelihood (1–5)	Severity (1–5)	Risk Score (L×S)	Mitigation Strategy	Testing Approach
GLM returns hallucinated reorder quantity (e.g. 10,000 units for an item that sold 20/week)	*3*	*5*	*15 (High)*	*Domain logic validator rejects quantities exceeding 3× the 30-day historical average; HTTP 422 returned, DB untouched*	*Seed test with known sales data, assert recommendation quantities fall within plausible bounds; inject mock GLM response with extreme value and confirm rejection*
GLM API unavailable or rate-limited mid-session	*3*	*4*	*12 (High)*	*3-attempt retry with simplified fallback prompt; graceful HTTP 422 error state; last cached result served*	*Mock GLM API timeout using pytest monkeypatch; assert retry triggers exactly 3×, then graceful error state returned without DB write*
Uploaded file has completely non-standard column names — pandas fails to parse	*4*	*3*	*12 (High)*	*GLM normalisation step maps arbitrary column names to schema before analysis; malformed rows dropped with user-visible count*	*Upload files with renamed/missing/shuffled columns; assert parsed record count is non-zero and quality warning shows correct dropped-row count*
Pydantic	*2*	*4*	*8*	*Review Pydantic*	*Run 20 real GLM*

© UMHackathon 2026

Email: umhackathon@um.edu.my

Technical Risk	Likelihood (1–5)	Severity (1–5)	Risk Score (L×S)	Mitigation Strategy	Testing Approach
schema validation fails on valid GLM output (false rejection)			*(Medium)*	*model definitions against actual GLM response format during development; widen acceptable types where needed*	*forecast calls, assert all pass Pydantic validation; log any failures for model correction*
OpenWeather Map API rate limit hit during demo	*3*	*2*	*6 (Medium)*	*Degrade gracefully — forecast proceeds without weather context; output notes absence to user*	*Mock weather API failure; assert forecast still returns with a user-visible "weather data unavailable" note in output*
Oversized file exceeds GLM context window — partial data processed silently	*2*	*5*	*10 (Medium)*	*Reject oversized payload entirely with HTTP 422 rather than truncating; user notified to reduce file size*	*Upload file >5MB and file with >context-window token equivalent; assert rejection before any DB write, no partial forecast stored*
SQLite write conflict under concurrent demo sessions	*2*	*3*	*6 (Medium)*	*SQLite WAL mode enabled; concurrent sessions scoped to separate session_ids*	*Simulate 5 concurrent upload+forecast requests; assert all sessions complete with correct session-scoped results, no cross-contamination*

Risk Assessment Scoring Criteria:

Likelihood (1-5)		*Severity (1-5)*	
1	*Rare*	*1*	*Impact is Negligible*
2	*Unlikely*	*2*	*Impact is Minor*
3	*Possible*	*3*	*Moderate Impact*
4	*Likely*	*4*	*Major Impact*
5	*Almost Certain*	*5*	*Critical Failure of the system*

Risk Score = Likelihood × Severity

Risk Level Reference:

© UMHackathon 2026

Email: umhackathon@um.edu.my

Risk Score	***Risk Level***	***Recommended Action***
1 – 5	*Low*	*Monitor only. Acceptable risks.*
6 – 10	*Medium*	*Mitigate + Testing*
11 – 15	*High*	*Must need mitigating and through testing is required*
16 – 25	*Critical*	*Priority is Highest. Need extensive level of testing.*

© UMHackathon 2026

Email: umhackathon@um.edu.my

3. Test Environment & Execution Strategy

Explain the testing environment. That means where the testing of your application will take place, how you are going to handle test data and the rules/threshold that you are incorporating to govern your testing phases.

- **Unit Test**
  - *Scope:*
    - *pandas parsing logic (column normalisation, row dropping, date extraction)*
    - *Pydantic schema validators (type enforcement, domain logic:*
    - *negative quantity rejection, confidence bounds 0–1)*
    - *limits.py token/character cap enforcement*
    - *GLM prompt template assembly (correct data injection, token budget not exceeded)*
    - *Data pre-aggregation logic (item + date grouping, summary compression)*
  - *Execution: pytest, run locally during development and triggered automatically in GitHub Actions on every push to any branch.*
  - *Isolation: GLM API, OpenWeatherMap API, SQLite, and file system are all mocked using pytest monkeypatch and unittest.mock. Only the unit under test executes real code.*
  - *Pass Condition: Happy path, negative cases (malformed input, type mismatch, out-of-bounds values), and edge cases (empty DataFrame, single-row file, all-zero quantities) all return correct output or raise expected exceptions with correct messages.*
- **Integration Test**
  - *Scope:*
    - *File upload endpoint → pandas parser → DS1 (uploads table write)*
    - *DS1 → data enrichment → GLM normalisation call → Pydantic validation → DS1 update*
    - *GLM forecast response → domain logic validation → DS2 write*
    - *DS2 + DS3 → P4 presentation → JSON response to frontend*
  - *Execution: Run after unit tests pass on PR merge to main. Real FastAPI endpoints tested via httpx TestClient. Real SQLite test DB. GLM API calls mocked to return fixture responses.*
  - *Workflow: Real API call chains tested end-to-end within the backend. Database reads/writes use a dedicated test SQLite instance, wiped between test runs.*
  - *Pass Condition: Data written to DS1 matches the parsed content of the uploaded file. Forecast written to DS2 matches the mocked GLM response after validation. JSON response to frontend contains all required fields (item_name, forecasted_demand, confidence_score, reasoning_text, reorder recommendations).*
- **Test Environment (CI/CD Practice):**
  - *Local Testing: Manual testing on localhost:8000 (FastAPI) + localhost:3000 (React). Developers run pytest before pushing.*

© UMHackathon 2026

Email: umhackathon@um.edu.my

- *Staging / CI: GitHub Actions triggers on every push. Runs unit tests + linting (ruff) automatically. PR to main additionally*

- *CI/CD Pipeline: Branch push → unit tests + lint check PR to main → unit tests + integration tests + coverage report Merge to main → full regression suite + AI output pass rate check*

- **Regression Testing & Pass/Fail Rules:**
  - *Execution Phase: Full regression suite runs on every merge to main, covering all 5 in-scope features: upload/parse, forecast, reorder, inventory optimisation, and What-If scenario.*

  - *Pass/Fail Condition: A test case passes only when the actual output exactly matches the expected output or falls within the defined tolerance (e.g. forecast quantities within ±5% of seeded expected values). Any mismatch is a fail and must be logged in the defect log before the next PR is reviewed.*

  - *Continuation Rule: Upload/parse tests (P1) must achieve 100% pass rate before forecast tests (P3) are executed — downstream tests are meaningless if the data layer is broken.*

- **Test Data Strategy:**
  - *Mock business datasets : milo.csv*
  - *Automated: pytest fixtures generate in-memory DataFrames with seeded sales patterns (known peaks on Fridays, low on Tuesdays) so forecast assertions can check for directionally correct predictions.*

- **Passing Rate Threshold:**
  - *Unit tests: 100% must pass (no exceptions)*
  - *Integration tests: 100% must pass before merge to main*
  - *Regression suite: Minimum 90% pass rate*
  - *AI output pass rate: Minimum 80% of documented prompt/response test pairs must pass*
  - *Critical (P0) tests: 100% — zero tolerance; any P0 failure blocks deployment*

© UMHackathon 2026

Email: umhackathon@um.edu.my

4. CI/CD Release Thresholds & Automation Gates

This section is mainly for defining metrics & threshold for success criteria. So, if a certain threshold is not met, the pipeline blocks the forward transition.

4.1 Integration Thresholds (Merging to Main)

The primary needs of this threshold to be crucial is because the main branch must be a stable source of truth. So, it needs to be achieved through continuous integration checks.

Checks	***Requirements***	***Project***	***Pass/Failed***
Automatic Build	***Zero Build Error***	***0 errors***	***Passed***
Unit Tests	***100% Passing Rate***	***100%***	***Passed***
Code Quality	***Zero Linting Error***	***0 errors***	***Passed***
Test Coverage	***Minimum 81.2%***	***82%***	***Passed***

© UMHackathon 2026

Email: umhackathon@um.edu.my

4.2 Deployment Thresholds (Pushing to Production)

Checks	***Requirements***	***Project***	***Pass/Failed***
Regression Test	***Minimum 90%***	***91%***	***Passed***
AI Output Pass Rate	***Minimum of 80% of documented prompt***	***78%***	***Failed***
Critical Bugs	***Zero P0/01 Bugs***	***0***	***Passed***
GLM pipeline latency	***Full pipeline < 15s at p95***	***~9s avg***	***Passed***
Individual GLM call latency	***< 8s per call at p95***	***3.2s avg***	***Passed***
Security	***API keys are not exposed***	***Clean***	***Passed***

© UMHackathon 2026

Email: umhackathon@um.edu.my

5. Test Case Specifications (Drafts)

Test Case ID	Test Type & Mapped Feature	Test Description	Test Steps	Expected Result	Actual Result
TC-01	*Happy Case — full end-to-end user journey* *Upload → Forecast → Reorder Recommendation*	*Verify that a business owner can upload a valid 30-day transactional CSV, receive a 7-day demand forecast with GLM reasoning, and receive a reorder shopping list with plain Malay/English explanations — all within the expected time threshold.*	*1. Open RetOps web app in browser* *2. Upload restaurant_30days.csv (valid 30-day sales data)* *3. Confirm data quality check passes with 0 rows dropped* *4. Observe forecast loading state appears immediately* *5. Wait for 7-day forecast chart to render 6. Verify GLM reasoning text is displayed below the chart 7. Navigate to Reorder Recommendation tab 8. Verify shopping list shows item name, quantity, recommended order date, and Malay/English explanation for each flagged item*	*Forecast chart renders with 7 data points per item. GLM reasoning text is non-empty and references the data (e.g. mentions day-of-week patterns or specific items). Reorder list contains at least 1 item with a complete explanation. Full pipeline completes within 15 seconds.*	*Same with the actual result* *Status: Passed*
TC-02	***Negative / Edge — Inventory Optimisation***	*Verify that uploading an inventory Excel file with no recognisable column*	*1. Prepare an Excel file with completely arbitrary column names (e.g.*	*System returns HTTP 422 or a user-facing error message such as "We could not*	*Status: Passed*

© UMHackathon 2026

Email: umhackathon@um.edu.my

Test Case ID	Test Type & Mapped Feature	Test Description	Test Steps	Expected Result	Actual Result
		headers does not crash the system and returns a user-readable error, not a 500.	*"col_a", "col_b", "col_c") and no date field 2. Upload the file as inventory data 3. Observe system response*	*identify the required columns in your file. Please ensure your file contains item name and stock quantity columns." No 500 error. No partial data written to DS1. User is offered guidance on expected file format.*	
TC-03	***Non-Functional — performance / load*** ***Full pipeline (upload → forecast → recommend)***	*Verify that the full RetOps pipeline (file upload through to recommendation delivery) completes within the 15-second threshold when 5 concurrent sessions run simultaneously, simulating hackathon demo conditions.*	*1. Prepare 5 identical copies of restaurant_30days.csv 2. Use locust or pytest-asyncio to fire 5 simultaneous POST /api/upload requests 3. Record end-to-end latency for each session (upload received to JSON response returned) 4. Record error rate*	*All 5 sessions complete within 15 seconds. Average latency < 12 seconds. Error rate = 0%. No cross-session data contamination (each session_id returns only its own forecast data).*	*Average Response time is received as more than 15 seconds.* *Status: Failed*

© UMHackathon 2026

Email: umhackathon@um.edu.my

6. AI Output & Boundary Testing (Drafts)

This section is designed to validate that your GLM integration does produce acceptable output and handles any inputs that are abnormal gracefully.

6.1. **Prompt/Response Test Pairs**

Document at least 3 Prompts/response test pairs. For each of them, do define the acceptance criteria - what a correct, acceptable or a failing response may look like for your use case.

Test ID	Prompt Input	Expected Output (Acceptance Criteria)	Actual Output	Status
AI-01	30-day sales CSV with consistent format. Prompt: "Generate 7-day demand forecast for all items."	Output is valid JSON matching Pydantic schema. forecasted_demand is a 7-element array of positive integers. reasoning_text is non-empty, references at least one item by name or mentions day-of-week pattern. confidence_score is between 0 and 1. No fabricated item names not present in the input.	The output is in JSON format which can match with pydantic schema.	Pass
AI-02	Valid sales CSV. What-If input: "Hari Raya is in 2 days."	Forecast output differs from baseline (no scenario) — at least one item shows higher predicted demand.	Reasoning text mentioned about holiday and weather.	Pass

© UMHackathon 2026

Email: umhackathon@um.edu.my

		`reasoning_text` explicitly mentions the holiday or seasonal event. Reorder quantities for high-velocity F&B items increase relative to baseline.		

6.2. Oversized/Larger Input Test

In this section, do define the maximum input size your system accepts. Do highlight here what happens when the limit is exceeded.

Fields	Details
Maximum Input Size	5,000 tokens of data context per GLM call (~400 transaction rows after aggregation) User prompt cap: 8,000 characters
Input used while testing	Upload a CSV with 5,000 raw rows (approx. 6–8× the recommended 30-day window for a 20-item catalogue)
Expected Behavior	System rejects the file before GLM call with HTTP 422 and a user-facing message: "Your file exceeds the maximum supported size. Please upload data covering the last 30 days only." No partial data written to DS1. No GLM API call is made.
Actual Behavior	The system allows user to upload file that exceeds max supported size
Status	Fail

© UMHackathon 2026

Email: umhackathon@um.edu.my

6.3. Adversarial/Edge Prompt Test

Prompt Input: What-If scenario field: "asdfgh jkl what if everything" (meaningless, no identifiable event or context) Expected Behavior: GLM either (a) asks for clarification in its output text, or (b) returns the baseline forecast unchanged with a note that the scenario was not interpretable. System does NOT crash. Schema validation still passes. No hallucinated event-driven demand spike is introduced.

Actual Behavior: The GLM ask for clarification
Status : Pass

6.4. Hallucinating Handling

Detection mechanism:
Layer 1 — Pydantic schema validation: type mismatches (e.g. string returned for forecasted_demand integer array) immediately raise a validation error before DB write.

Layer 2 — Domain logic validation: custom validators check: - forecasted_demand values must all be >= 0 - confidence_score must be strictly between 0.0 and 1.0 - reorder_qty must not exceed 3× the item's 30-day historical average (sanity cap) - Item names in recommendations must exist in the uploaded transaction data (no fabricated products)

Layer 3 — Self-correction loop: if any layer fails, the exact validation error is injected back into a retry prompt instructing GLM to correct the specific violation. Maximum 3 retry attempts at temperature=0.

Layer 4 — Graceful rejection: after 3 failed retries, HTTP 422 is returned to the frontend. Database remains untouched. Failure is logged with error code for developer review. No partial or unvalidated output is ever shown to the user.

© UMHackathon 2026

Email: umhackathon@um.edu.my